SESSION 7: CONDITIONAL STATEMENTS

SESSION OVERVIEW:

In this session, the students will be able to:

- Understand what control statements are
- Understand the importance of control statements
- Understand the types of control statements
- Understand what conditional statements are
- Understand what nested conditional statements are
- Understand conditional statements with shorthand notations

KEY POINTS AND EXAMPLES:

Understand what control statements are

Control statements in Python are like traffic signals for your code. They help decide which parts of your program should run and when.

For example, think of a traffic light. When it's green, cars can go. When it's red, they stop. Similarly, in Python, we have statements like if, else, and elif that help us make decisions based on conditions.

We also have loops, like for and while, which are like repeating patterns. They allow us to do things multiple times without writing the same code over and over.

Lastly, we have special statements like break, continue, and pass that let us control the flow of our program in unique ways.

Understand the importance of control statements

1. **Making Decisions:** Control statements like if, else, and elif help us make decisions based on certain conditions. For example, we might want our program to do one thing if a condition is true, and another thing if it's false.

2. **Repeating Tasks:** Loops such as for and while allow us to repeat certain tasks multiple times without having to write the same code over and over again. This is incredibly useful for automating repetitive tasks or processing large amounts of data.

3. **Handling Exceptions:** Control statements also help us handle exceptional situations gracefully. We can use them to deal with errors, interruptions, or unexpected events in our code, ensuring that our program can continue running smoothly even when things don't go as planned.

4. **Creating Algorithms:** Many algorithms and problem-solving techniques rely on control statements to control the flow of execution and make decisions at each step. Whether it's sorting a list, searching for an item, or traversing a data structure, control statements are essential for implementing these algorithms effectively.

Understand the types of control statements

The three main types of control statements in Python are:

1. **Conditional Statements (if, elif, else):** Conditional statements are used to execute different blocks of code based on whether a certain condition is true or false.

2. **Looping Statements (for, while):** Looping statements are used to execute a block of code repeat. (will cover later)

3. **Control Transfer Statements (break, continue, pass):** Transfer statements are used to alter the flow of execution within loops or conditional statements. (will cover later)

As we discussed previously, indentation (:) in Python is used to define the structure of code blocks.

Understand what conditional statements are

The primary conditional statement in Python is the if statement, which can be supplemented with elif (short for "else if") and else statements for additional conditions and default actions.

An "if statement" is written by using the if keyword.

1. **if Statement:** The if statement is used to execute a block of code if a specified condition is true.

Syntax:

```
if condition:
    # Code block to execute if condition is true
```

- if condition: or if (condition): are the correct syntax for an if statement
- The condition can be any expression that evaluates to either True or False.

- If the condition is true, the indented code block immediately following the if statement is executed. If the condition is false, the code block is skipped and The program continues to execute the code outside the if block.

```
x = 10

if x > 0:
    print("x is positive")
```

In this example, the condition x > 0 is evaluated. Since x is indeed greater than 0, the statement "x is positive" is printed.

```
a = 33
b = 200

if b > a:
  print("b is greater than a")
```

In this example we use two variables, a and b, which are used as part of the if statement to test whether b is greater than a. As a is 33, and b is 200, we know that 200 is greater than 33, and so we print to screen that "b is greater than a"

```
# Define a variable
x = -5

# Check if x is positive
if x > 0:
    print("x is positive")
    print("This line is also inside the if block")  # This line is
part of the if block

# This line is outside the if block
print("This line is outside the if block")

# Output:
# This line is outside the if block
```

In this example:

- The variable x is assigned the value -5.
- The if statement checks if x is greater than 0. Since -5 is not greater than 0, the condition evaluates to False.

- Therefore, the code block associated with the if statement (indented lines) is skipped entirely.
- The program continues to execute the code outside the if block, and the line "This line is outside the if block" is printed to the console.

In Python, when you use a number in a conditional statement like if, Python evaluates that number in a boolean context.

- Any non-zero number is considered True.
- Zero is considered False.

```
if -1:
    print("Negative numbers are considered True.")
```

The output will be:

```
Negative numbers are considered True.
```

So, when you write if -1:, Python evaluates -1 as a boolean value. Since -1 is a non-zero number, it's considered True in Python's boolean evaluation. Therefore, the condition if -1: is True, and the message "Negative numbers are considered True." is printed.

If statement, without indentation (will raise an error):

```
a = 33
b = 200

if b > a:
print("b is greater than a") # you will get an error
```

The output will be:

```
print("b is greater than a")
        ^
IndentationError: expected an indented block
```

This error occurs because Python expects an indented block of code after the if statement, but since the print("b is greater than a") statement is not indented, Python raises an IndentationError.

```
x = 5

if x = 5:
    print("x is equal to 5")
```

The output will be:

```
SyntaxError: invalid syntax
```

In this code snippet, we intended to check if the variable x is equal to 5. However, instead of using the comparison operator ==, we mistakenly used the assignment operator =. This will result in a syntax error because the if statement expects a boolean expression (a condition), but it encounters an assignment statement instead.

To fix this error, we should use the comparison operator == to compare the value of x with 5, like this:

```
x = 5

if x == 5:
    print("x is equal to 5")
```

Now, the code will execute without any errors, and the message "x is equal to 5" will be printed if the condition is true.

2. **if-else Statement:** It is an optional part of the if statement. It allows you to execute a block of code when the condition of the if statement evaluates to false. If the condition of the if statement is not met, then the code block associated with the else statement is executed.

Syntax:

```
if condition:
    # Code block to execute if condition is true
else:
    # Code block to execute if condition is false
```

If the condition is true, the indented code block immediately following the if statement is executed. If the condition is false, the code block following the else statement is executed.

```
x = 10

if x > 15:
```

```
    print("x is greater than 15")
else:
    print("x is not greater than 15")
```

The output will be:

```
x is not greater than 15
```

The variable x is assigned the value 10. The if statement checks if x is greater than 15. Since 10 is not greater than 15, the condition evaluates to false.Therefore, the code block associated with the else statement is executed, and the message "x is not greater than 15" is printed to the console.

```
number = 10
# Check if the number is a multiple of 2
if number % 2 == 0:
    print("Multiple of 2")
else:
    print("Not a Multiple of 2")
```

The output will be:

```
Multiple of 2
```

The answer "Multiple of 2" is printed because the input number, which is 10 in this case, is evenly divisible by 2 without any remainder. Therefore, it satisfies the condition for being a multiple of 2.

```
x = "5"
print("Type of x:", type(x))
if x == 5:
     print("x is 5")
else:
     print("x is not 5 - check the type of x!")
```

The output will be:

```
Type of x: <class 'str'>
x is not 5 - check the type of x!
```

In this example,

- Initially, the variable x is assigned the value "5", which is a string.
- When the type of x is printed using type(x), it confirms that x is indeed a string.
- In the if statement, x is compared with the integer 5, but since they have different types (str and int), the condition evaluates to False.
- Therefore, the else block is executed, printing the message "x is not 5 - check the type of x!".

```
# Dictionary
my_dict = {'a': 1, 'b': 2, 'c': 3}

# Key to check
key_to_check = 'b'

# Check if the key exists in the dictionary
if key_to_check in my_dict:
    print("yes")
else:
    print("no")
```

The output will be:

```
yes
```

This program checks if the key specified in key_to_check exists in the dictionary my_dict. If the key exists, it prints "yes". Otherwise, it prints "no".

```
if 0:
     print("This will not print.")
else:
     print("Zero is considered False.")
```

The output will be:

```
Zero is considered False.
```

In Python, zero is considered False in a boolean context. Therefore, when you use if 0:, Python evaluates 0 as a boolean value, which is False. As a result, the code block under the else statement is executed, printing "Zero is considered False." to the console.

```
else:
    print("x is not greater than 10")

print("End of program")
```

you'll encounter a SyntaxError because an else statement cannot exist on its own without a corresponding if statement.

Similarly, if the else statement is not properly indented, you may encounter an IndentationError.

```
if x > 10:
    print("x is greater than 10")
else:
print("x is not greater than 10")  # Incorrect indentation

print("End of program")
```

3. **if-elif-else Statement:** The elif statement allows you to check multiple conditions after the initial if statement. The if-elif-else statement in Python allows you to evaluate multiple conditions sequentially and execute different blocks of code based on the first condition that evaluates to true. This construct is useful when you have several conditions to check and want to execute specific code blocks based on the outcome of those conditions.

Syntax:

```
if condition1:
    # Code block to execute if condition1 is true
elif condition2:
    # Code block to execute if condition1 is false and condition2
is true
elif condition3:
    # Code block to execute if condition1 and condition2 are false
and condition3 is true
...
else:
    # Code block to execute if all conditions are false
```

- The if statement checks the first condition. If it is true, the corresponding code block is executed, and the rest of the elif and else blocks are skipped.
- If the first condition is false, the next elif statement is evaluated. If it is true, the

corresponding code block is executed, and subsequent elif and else blocks are skipped.

- This process continues until either one of the conditions evaluates to true, in which case the corresponding code block is executed, or the else block is reached if none of the conditions are true.

```
x = 10

if x > 10:
    print("x is greater than 10")
elif x == 10:
    print("x is equal to 10")
else:
    print("x is less than 10")
```

The output will be:

```
x is equal to 10
```

In this example, the first condition x > 10 is false, so the elif statement x == 10 is evaluated. Since x is indeed equal to 10, the corresponding code block print("x is equal to 10") is executed.
The else block is not executed because the condition x == 10 was true.

```
# Define three variables
a = 10
b = 5
c = 8
# Check if 'a' is greater than both 'b' and 'c'
if a > b and a > c:
    print("a is greater than both b and c")
# Check if 'b' is greater than both 'a' and 'c'
elif b > a and b > c:
    print("b is greater than both a and c")
# Check if 'c' is greater than both 'a' and 'b'
elif c > a and c > b:
    print("c is greater than both a and b")
else:
    print("None of the conditions are met")
```

The output will be:

```
a is greater than both b and c
```

```
x = 4

if x > 5:
    print("x is greater than 5")
elif x < 5:
    print("x is less than 5")
elif x == 5:
    print("x is equal to 5")
```

The output will be:

```
x is less than 5
```

- The first if statement checks if x is greater than 5, which is not true (4 is less than 5), so the corresponding block is skipped.
- The first elif statement checks if x is less than 5, which is true (4 is less than 5), so the corresponding block is executed and "x is less than 5" is printed.
- The second elif statement and any subsequent statements are not evaluated because the condition in the previous elif statement was true.

If none of the conditions are true, no code block is executed.

Using if and elif without an else statement is perfectly valid and can be useful in situations where you only need to execute code based on specific conditions without providing a default action.

```
x = 10

elif x > 5:
    print("x is greater than 5")
elif x < 5:
    print("x is less than 5")
```

The output will be:

```
SyntaxError: invalid syntax
```

In this example, the elif statement is used without a preceding if statement. This will result in a SyntaxError because elif must always follow an if statement.

Understand what nested conditional statements are

Nested statements in Python refer to the practice of including one statement within another, such as placing one conditional statement inside another conditional statement.

1. **Nested if Statements:** Nested if statements in Python involve placing one if statement inside another if statement. This allows for hierarchical conditional evaluation, where inner conditions are checked only if outer conditions are true.

Syntax:

```
if condition1:
    # Code block to execute if condition1 is true
    if condition2:
        # Code block to execute if both condition1 and condition2
are true
```

```
x = 10
y = 5

if x > 0:
    print("x is positive")
    if y > 0:
        print("y is positive")
        print("Both x and y are positive")
```

In this example:

- The outer if statement checks if x is positive.
- If x is positive, the inner if statement checks if y is positive.

```
x = -5
y = 5

if x > 0:
    print("x is positive")
    if y > 0:
        print("y is positive")
        print("Both x and y are positive")
```

- The outer if statement checks if x is positive. Since x is -5 (not greater than 0), this condition is not true. Therefore, the code block associated with the outer if statement is skipped entirely.
- The program moves to the next line of code after the outer if block. Since the outer if condition is not true, the nested if statement and its associated code block are never executed. Therefore, there is no output from the nested if block in this case.

2. **Nested if-else Statements:** Nested if-else statements in Python involve placing one if-else statement inside another if or else block.

Syntax:

```
if condition1:
    # Code block to execute if condition1 is true
    if condition2:
        # Code block to execute if condition2 is true
    else:
        # Code block to execute if condition2 is false
else:
    # Code block to execute if condition1 is false
```

In this syntax:

- If condition1 is true, the inner if statement is evaluated.
- If condition2 is true, the corresponding code block executes.
- If condition2 is false, the else block associated with the inner if statement executes.
- If condition1 is false, the else block associated with the outer if statement executes.

```
if condition:
    # Code block to execute if condition is true
else:
    # Code block to execute if condition is false
    if another_condition:
        # Code block to execute if another_condition is true
    else:
        # Code block to execute if another_condition is false
```

In this syntax:

- The initial if statement checks a condition.
- If the condition is true, the code block associated with the if statement is executed.
- If the condition is false, the code block associated with the else statement is executed.

- Within the else block, you can nest another if-else structure to further refine the logic based on additional conditions.

```
x = 10
y = 5

if x > 0:
    print("x is positive")
    if y > 0:
        print("y is positive")
        print("Both x and y are positive")
    else:
        print("y is non-positive")
        print("x is positive, but y is non-positive")
else:
    print("x is non-positive")
```

The output will be:

```
x is positive
y is positive
Both x and y are positive
```

```
x = -10
y = 5

if x > 0:
    print("x is positive")
else:
    print("x is non-positive")
    if y > 0:
        print("y is positive")
    else:
        print("y is non-positive")
```

The output will be:

```
x is non-positive
y is positive
```

3. **Nested if-elif-else Statements:** Nested if-elif-else statements in Python allow for the organization of complex conditional logic within code.

Syntax:

```
if condition1:
    # Code block executed if condition1 is true
    if condition2:
        # Code block executed if both condition1 and condition2
are true
    elif condition3:
        # Code block executed if condition1 is true, condition2 is
false, but condition3 is true
    else:
        # Code block executed if condition1 is true, and both
condition2 and condition3 are false
elif condition4:
    # Code block executed if condition1 is false, but condition4
is true
else:
    # Code block executed if all conditions are false
```

In this syntax:

- The outer if statement checks condition1.
- If condition1 is true, it evaluates the inner if-elif-else structure.
- If condition2 is true, the corresponding code block executes.
- If condition2 is false and condition3 is true, the corresponding code block executes.
- If both condition2 and condition3 are false, the else block executes.
- If condition1 is false, it evaluates the next elif condition (condition4).
- If condition4 is true, the corresponding code block executes.
- If none of the conditions are true, the else block at the end executes.

```
age = 25
income = 50000

if age >= 18:
    print("You are eligible to vote.")
    if income >= 30000:
        print("You are eligible to apply for a credit card.")
```

```
    elif income >= 20000:
        print("You are eligible to apply for a personal loan.")
    else:
        print("You are eligible to open a savings account.")
else:
    print("You are not eligible to vote yet.")
```

This code checks if a person is eligible to vote based on their age, and if they are, it further checks their income to determine additional financial eligibility options. If they are not eligible to vote yet, it prints a message accordingly.

The output will be:

```
You are eligible to vote.
You are eligible to apply for a credit card.
```

Understand conditional statements with shorthand notations

In Python, we can use shorthand notations to simplify the syntax of conditional statements, making our code more concise and readable.

1. **Shorthand if Statement:** If you have only one statement to execute, you can put it on the same line as the if statement.

Syntax:

```
if a > b: print("a is greater than b")
```

```
age = 25

# Print a message if age is greater than or equal to 18
if age >= 18: print("You are an adult.")
```

2. **Shorthand if-else Statement:** the shorthand if-else statement allows us to assign a value to a variable based on a condition, but with a different value for each outcome. The technique of using shorthand if-else statements in Python is commonly referred to as "Ternary Operators" or "Conditional Expressions."

Syntax:

```
result = value1 if condition else value2
```

```
text = "Hello, world!"

# Check if the text contains the word "world"
contains_world = True if "world" in text else False
print(contains_world) # Output: True
```

In this example, we have a text variable containing the string "Hello, world!". We use a shorthand if-else statement to check if the substring "world" is present in the text. If the condition "world" in text evaluates to True, then the variable contains_world is assigned the value True; otherwise, it's assigned the value False.

```
x = 10
y = 20

# Assign the maximum of x and y to result
result = x if x > y else y # Output: 20
```

In this example, we have two variables x and y with values 10 and 20, respectively. We use a shorthand if statement to assign the maximum of x and y to the variable result. The condition x > y checks if x is greater than y. If the condition is True, result is assigned the value of x; otherwise, it's assigned the value of y.

```
x = 10
message = print("Even") if x % 2 == 0
```

The output will be:

```
SyntaxError: invalid syntax
```

when using the shorthand if statement without the else part, we need to provide a default value in case the condition evaluates to False.

```
x = 10
message = "Even" if x % 2 == 0 else "Odd"
```

3. **Shorthand if-elif-else Statement:** While there's no direct shorthand for if-elif-else statements, we can achieve a similar effect using nested shorthand if-else statements. In Python, expressions are evaluated from left to right.

Syntax:

```
result = (value1 if condition1 else value2) if condition2 else
value3
```

In this syntax,

In the expression result = (value1 if condition1 else value2) if condition2 else value3, Python evaluates expressions from left to right.

- The leftmost part of the expression, (value1 if condition1 else value2), is evaluated first.
- If condition1 is True, it assigns value1 to the result of the inner expression.
- If condition1 is False, it assigns value2 to the result of the inner expression.
- If the result of the inner expression is False (i.e., value2 was assigned), Python then moves to the outer if-else statement to evaluate condition2.
- If condition2 is True, the result of the outer if-else statement will be the result of the inner expression (which is value2 in this case).
- If condition2 is False, Python assigns value3 to the variable result.

Note: When using parentheses ( ), the expression inside the parentheses is considered the inner expression.

```
value1 = "Python"
value2 = "Java"
value3 = "JavaScript"
condition1 = True
condition2 = True

# Expression to evaluate
result = (value1 if condition1 else value2) if condition2 else
value3

# Print the result
print("Result:", result) # Output: Python
```

In this example,

- both condition1 and condition2 are True. Since condition1 is True, the inner expression (value1 if condition1 else value2) evaluates to value1, which is "Python".

```
value1 = "Python"
value2 = "Java"
value3 = "JavaScript"
condition1 = False
condition2 = True
```

```
# Expression to evaluate
result = (value1 if condition1 else value2) if condition2 else
value3

# Print the result
print("Result:", result) # Output: Java
```

In this example,

- condition1 is False and condition2 is True. Since condition1 is False, the inner expression (value1 if condition1 else value2) evaluates to value2, which is "Java".
- The outer condition condition2 is True, so result is assigned the value of the inner expression, which is "Java".

```
# Determine the grade based on the score
score = 75

# Nested shorthand if-else statement to determine the grade
grade = 'A' if score >= 90 else ('B' if score >= 80 else ('C' if
score >= 70 else 'D'))

# Print the grade
print("Grade:", grade) # Output: C
```

In this example:

- If the score is greater than or equal to 90, the grade is 'A'.
- Otherwise, if the score is greater than or equal to 80, the grade is 'B'.
- Otherwise, if the score is greater than or equal to 70, the grade is 'C'.
- Otherwise, the grade is 'D'.

Note: Python evaluates expressions from left to right.

```
a = 330
b = 330

print("A") if a > b else print("=") if a == b else print("B") #
Output: =
```

- The outer if-else statement checks if a is greater than b. If true, it prints "A".

- If the outer condition is false, it moves to the next level of the nested structure, where it checks if a is equal to b. If true, it prints "=".
- If neither of the above conditions is true, it prints "B".

This code snippet is equivalent to the following nested structure:

```
if a > b:
    print("A")
else:
    if a == b:
        print("=")
    else:
        print("B")
```